In [1]:

```
%pip install pandas numpy matplotlib scikit-learn
```

```
Requirement already satisfied: pandas in c:\users\hanis\anaconda3\lib\site-packages (2.
3.3)
Requirement already satisfied: numpy in c:\users\hanis\anaconda3\lib\site-packages (2.3.
5)
Requirement already satisfied: matplotlib in c:\users\hanis\anaconda3\lib\site-packages
(3.10.6)
Requirement already satisfied: scikit-learn in c:\users\hanis\anaconda3\lib\site-package
s (1.7.2)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\hanis\anaconda3\lib\si
te-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\hanis\anaconda3\lib\site-package
s (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\hanis\anaconda3\lib\site-packa
ges (from pandas) (2025.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\hanis\anaconda3\lib\site-pac
kages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\users\hanis\anaconda3\lib\site-package
s (from matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\hanis\anaconda3\lib\site-pa
ckages (from matplotlib) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\hanis\anaconda3\lib\site-pa
ckages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in c:\users\hanis\anaconda3\lib\site-pack
ages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in c:\users\hanis\anaconda3\lib\site-packages
(from matplotlib) (12.0.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\hanis\anaconda3\lib\site-pac
kages (from matplotlib) (3.2.5)
Requirement already satisfied: scipy>=1.8.0 in c:\users\hanis\anaconda3\lib\site-package
s (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in c:\users\hanis\anaconda3\lib\site-packag
es (from scikit-learn) (1.5.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\hanis\anaconda3\lib\site
-packages (from scikit-learn) (3.5.0)
Requirement already satisfied: six>=1.5 in c:\users\hanis\anaconda3\lib\site-packages (f
rom python-dateutil>=2.8.2->pandas) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

In [6]:

```
# ----------------------------------------Task 4-------------------------------------
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error, mean_absolute_error
from sklearn.model_selection import train_test_split

# Create dataset using Pandas
data = {
    'Location': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Response': [2, 4, 5, 4, 6, 7, 8, 8, 10, 12]
}
df = pd.DataFrame(data)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
x = df[['Location']]
y = df['Response']

# Split the data
x1, x2, y1, y2 = train_test_split(
    x, y, test_size=0.2, random_state=42
)

# Polynomial features
poly = PolynomialFeatures(degree=2)
xp1 = poly.fit_transform(x1)
xp2 = poly.transform(x2)

# Linear Regression
a = LinearRegression()
a.fit(x1, y1)
p1 = a.predict(x2)

# Polynomial Regression
b = LinearRegression()
b.fit(xp1, y1)
p2 = b.predict(xp2)

# Ridge Regression
c = Ridge(alpha=1.0)
c.fit(xp1, y1)
p3 = c.predict(xp2)

# LASSO Regression
d = Lasso(alpha=0.01, max_iter=10000)
d.fit(xp1, y1)
p4 = d.predict(xp2)

# Error measures
print("Linear MSE:", round(mean_squared_error(y2, p1), 4))
print("Polynomial MSE:", round(mean_squared_error(y2, p2), 4))
print("Ridge MSE:", round(mean_squared_error(y2, p3), 4))
print("LASSO MSE:", round(mean_squared_error(y2, p4), 4))
print("Polynomial MAE:", round(mean_absolute_error(y2, p2), 4))

# Plot regression curves
z = pd.DataFrame({'Location': np.linspace(1, 10, 100)})
zp = poly.transform(z)

plt.subplot(1, 2, 1)
plt.scatter(x, y, color='black', label='Actual')
plt.plot(z, a.predict(z), label='Linear')
plt.plot(z, b.predict(zp), label='Polynomial')
plt.plot(z, c.predict(zp), label='Ridge')
plt.plot(z, d.predict(zp), label='LASSO')
plt.xlabel("Location")
plt.ylabel("Response")
plt.title("Regression Models")
plt.legend()

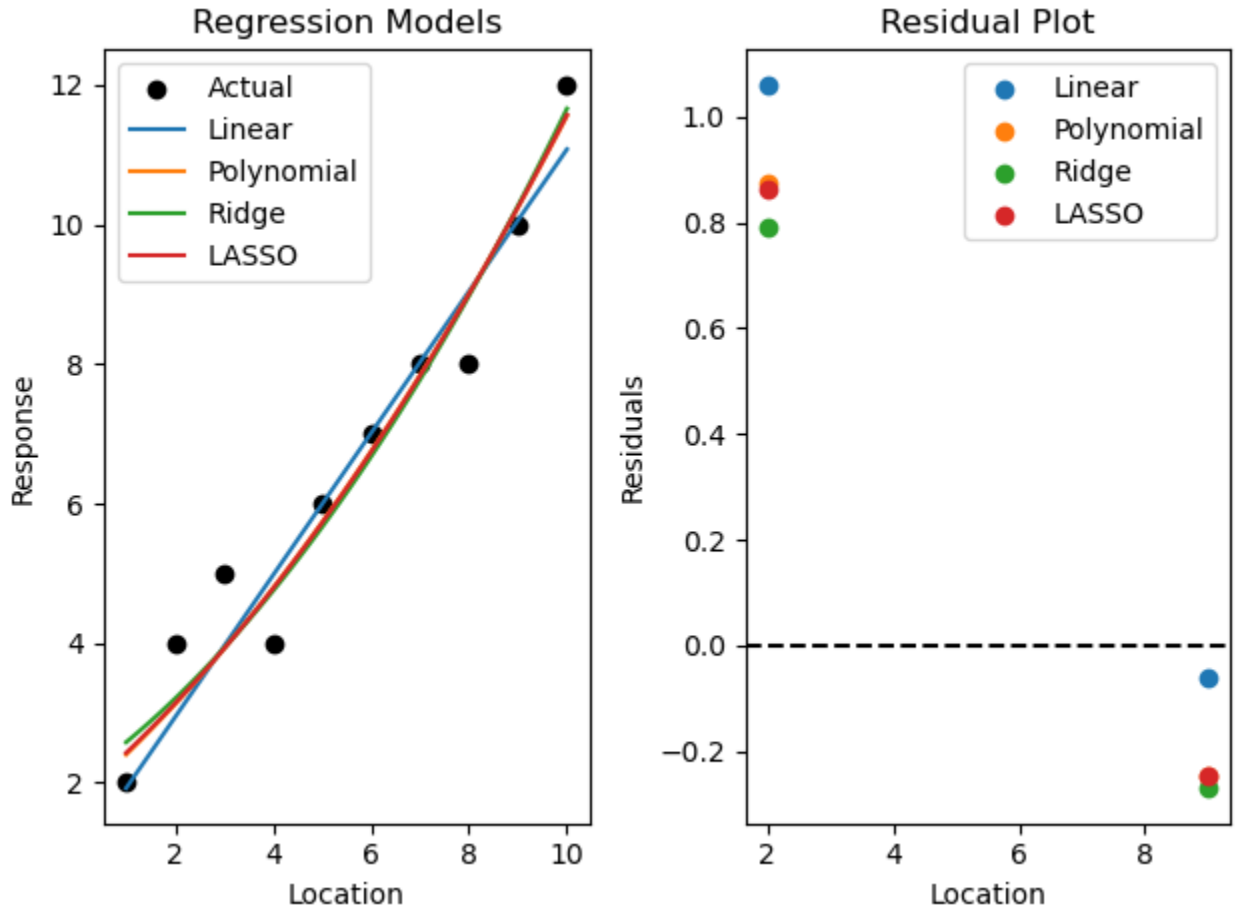
# Plot residuals
plt.subplot(1, 2, 2)
plt.scatter(x2['Location'], y2 - p1, label='Linear')
plt.scatter(x2['Location'], y2 - p2, label='Polynomial')
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
plt.scatter(x2['Location'], y2 - p3, label='Ridge')
plt.scatter(x2['Location'], y2 - p4, label='LASSO')
plt.axhline(0, color='black', linestyle='--')
plt.xlabel("Location")
plt.ylabel("Residuals")
plt.title("Residual Plot")
plt.legend()

plt.tight_layout()
plt.show()
```

```
Linear MSE: 0.564
Polynomial MSE: 0.4136
Ridge MSE: 0.3505
LASSO MSE: 0.4035
Polynomial MAE: 0.5603
```



In [7]:

```
# ----------------------------Task 5-------------------------------------------
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

print("Original shape:", X.shape)

# PCA
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

print("PCA reduced shape:", X_pca.shape)
print("PCA variance retained:", round(sum(pca.explained_variance_ratio_), 4))

# LDA
lda = LDA(n_components=2)
X_lda = lda.fit_transform(X, y)

print("LDA reduced shape:", X_lda.shape)

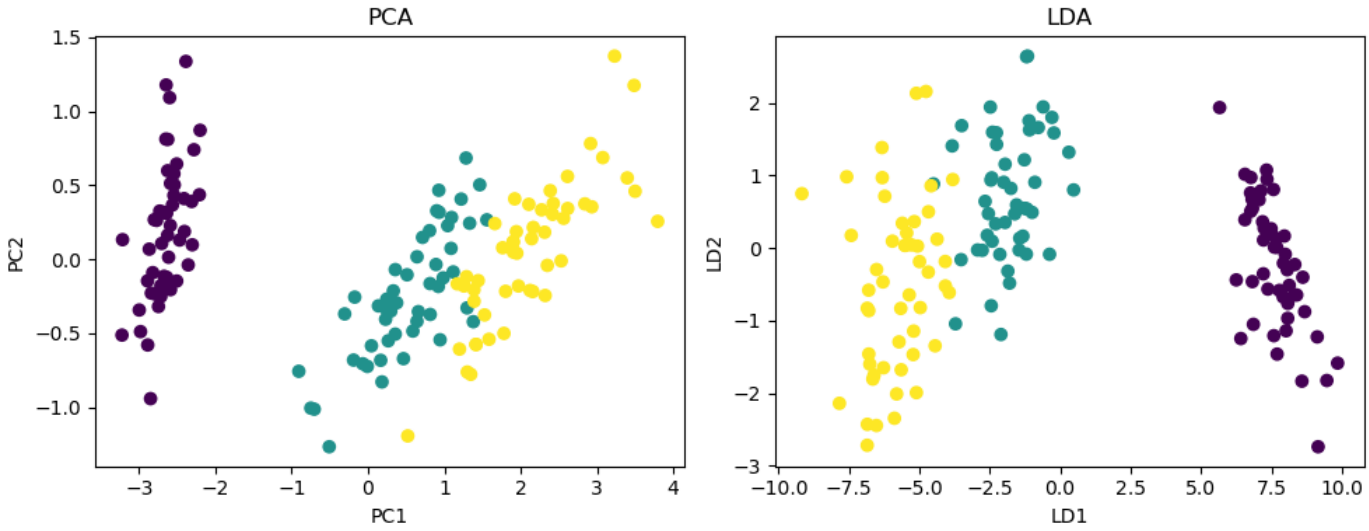
# Plot results
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='viridis')
plt.title("PCA")
plt.xlabel("PC1")
plt.ylabel("PC2")

plt.subplot(1, 2, 2)
plt.scatter(X_lda[:, 0], X_lda[:, 1], c=y, cmap='viridis')
plt.title("LDA")
plt.xlabel("LD1")
plt.ylabel("LD2")

plt.tight_layout()
plt.show()
```

```
Original shape: (150, 4)
PCA reduced shape: (150, 2)
PCA variance retained: 0.9777
LDA reduced shape: (150, 2)
```



In [10]:

```
# -------------------------task 6---------------------------------
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 1. Load dataset
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42
)

# 2. Initialize and train the K-NN classifier (setting K = 3)[cite: 5]
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)

# 3. Predict on test data and check accuracy[cite: 5]
y_pred = knn.predict(X_test)
print("Predictions:", y_pred)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Predictions: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Accuracy: 1.0
```

In [12]:

```
# -------------------------task 6 lab format--------------------------------
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

# 1. Load dataset
iris = load_iris()
X = iris.data
y = iris.target

print("K-NEAREST NEIGHBORS CLASSIFICATION")
print("----------------------------------")
print("Dataset Name:", "Iris")
print("Total Samples:", len(X))
print("Total Features:", X.shape[1])
print("Classes:", iris.target_names)

# 2. Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("\nDATASET SPLITTING")
print("-----------------")
print("Training Samples:", len(X_train))
print("Testing Samples:", len(X_test))

# 3. Train K-NN classifier
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)

print("\nMODEL DETAILS")
print("-------------")
print("Algorithm: K-Nearest Neighbors")
print("K Value:", 3)

# 4. Predict test data
y_pred = knn.predict(X_test)

print("\nPREDICTION RESULTS")
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
print("------------------")
print("Actual Values:   ", y_test)
print("Predicted Values:", y_pred)

# 5. Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nMODEL EVALUATION")
print("----------------")
print("Accuracy:", round(accuracy * 100, 2), "%")
```

```
K-NEAREST NEIGHBORS CLASSIFICATION
--------------------------------
Dataset Name: Iris
Total Samples: 150
Total Features: 4
Classes: ['setosa' 'versicolor' 'virginica']

DATASET SPLITTING
-----------------
Training Samples: 120
Testing Samples: 30

MODEL DETAILS
-------------
Algorithm: K-Nearest Neighbors
K Value: 3

PREDICTION RESULTS
------------------
Actual Values:    [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Predicted Values: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]

MODEL EVALUATION
----------------
Accuracy: 100.0 %
```

In [4]:

```
# -------------------------------Task 7-------------------------------
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import make_pipeline

# 1. Load dataset
df = pd.read_csv(r"C:\Users\hanis\OneDrive\Desktop\Hanish\DAA\german_credit.csv")

# print("GERMAN CREDIT DATASET")
# print("---------------------")
# print("Dataset Shape:", df.shape)
# print("\nFirst 5 Records:")
# print(df.head())

# 2. Separate features and target
X = df.drop('default', axis=1)
y = df['default']
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```python
# 3. Identify numerical and categorical columns
num_cols = X.select_dtypes(include='number').columns
cat_cols = X.select_dtypes(exclude='number').columns

# 4. Convert categorical data and scale numerical data
preprocessor = ColumnTransformer([
    ('num', StandardScaler(), num_cols),
    ('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols)
])

# 5. Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

print("\nTraining Samples:", len(X_train))
print("Testing Samples:", len(X_test))

# 6. Create and train Logistic Regression model
model = make_pipeline(
    preprocessor,
    LogisticRegression(max_iter=3000)
)
model.fit(X_train, y_train)

# 7. Predict credit status
y_pred = model.predict(X_test)

print("\nCREDIT STATUS CLASSIFICATION")
print("----------------------------")
print("Actual Values:   ", y_test.to_numpy())
print("Predicted Values:", y_pred)

# 8. Calculate confusion matrix and accuracy
cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

print("\nModel Accuracy:", round(accuracy * 100, 2), "%")
```

```
Training Samples: 700
Testing Samples: 300

CREDIT STATUS CLASSIFICATION
----------------------------
Actual Values:    [0 0 0 0 0 0 0 0 0 1 1 0 0 1 0 1 0 1 0 0 1 0 0 0 1 0 0 0 0 1 0 1 1 0 0
1 0
 0 0 0 0 1 0 1 0 0 0 0 1 1 1 1 1 0 1 1 1 0 0 1 1 0 1 0 1 0 0 1 0 0 0 0 0 0
 1 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 1 0 1 1 0 0 0 0 1 0 0
 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 1 1 0 0 0 0 0 1 0 1 1 0 1 1 0
 0 0 0 0 0 0 1 0 0 0 0 1 0 1 1 0 0 0 1 0 0 0 0 0 1 0 1 0 1 0 0 1 0 0 1 0 0
 0 0 0 0 0 0 1 0 1 1 0 1 1 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 1 0 0 0 0
 0 0 1 0 1 1 1 0 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 0 1 1 0 0 0 1 0 1 0 0
 1 0 1 0 0 0 1 0 1 0 0 1 1 1 0 1 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 1 0 1
 1 0 0 0]
Predicted Values: [0 0 0 0 0 0 0 0 0 1 1 1 0 1 1 1 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
1 0
 0 0 0 1 0 0 0 0 0 0 0 1 0 0 1 0 0 1 1 0 0 1 0 1 1 1 0 0 0 0 0 0 1 0 0 0 0
 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0
 0 1 1 0 0 0 1 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0 1 0 0 0 1 0 1 1 1 0 1 0
 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0
 0 0 1 0 0 1 1 0 1 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 1 0 0 1 0 1 0 1 1 0 0 0 0
 0 0 1 0 0 1 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 1 0 0 0 0
 0 0 1 0 0 0 0 0 1 1 0 0 1 1 0 0 0 1 0 0 1 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0
 0 0 0 0]

Confusion Matrix:
[[187  23]
 [ 42  48]]

Model Accuracy: 78.33 %
```

In [14]:

```python
# ------------------------------Task 8----------------------------------
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Load dataset
df = pd.read_csv(r"C:\Users\hanis\OneDrive\Desktop\Hanish\DAA\task8.csv")

# Correlation matrix a nd heatmap
numeric_df = df.select_dtypes(include='number')
corr = numeric_df.corr()

sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f')
plt.title("Correlation Heatmap")
plt.show()

# Features and target
X = numeric_df.drop(['SOLD PRICE'],axis=1)
y = numeric_df['SOLD PRICE']

X = X.fillna(X.mean())

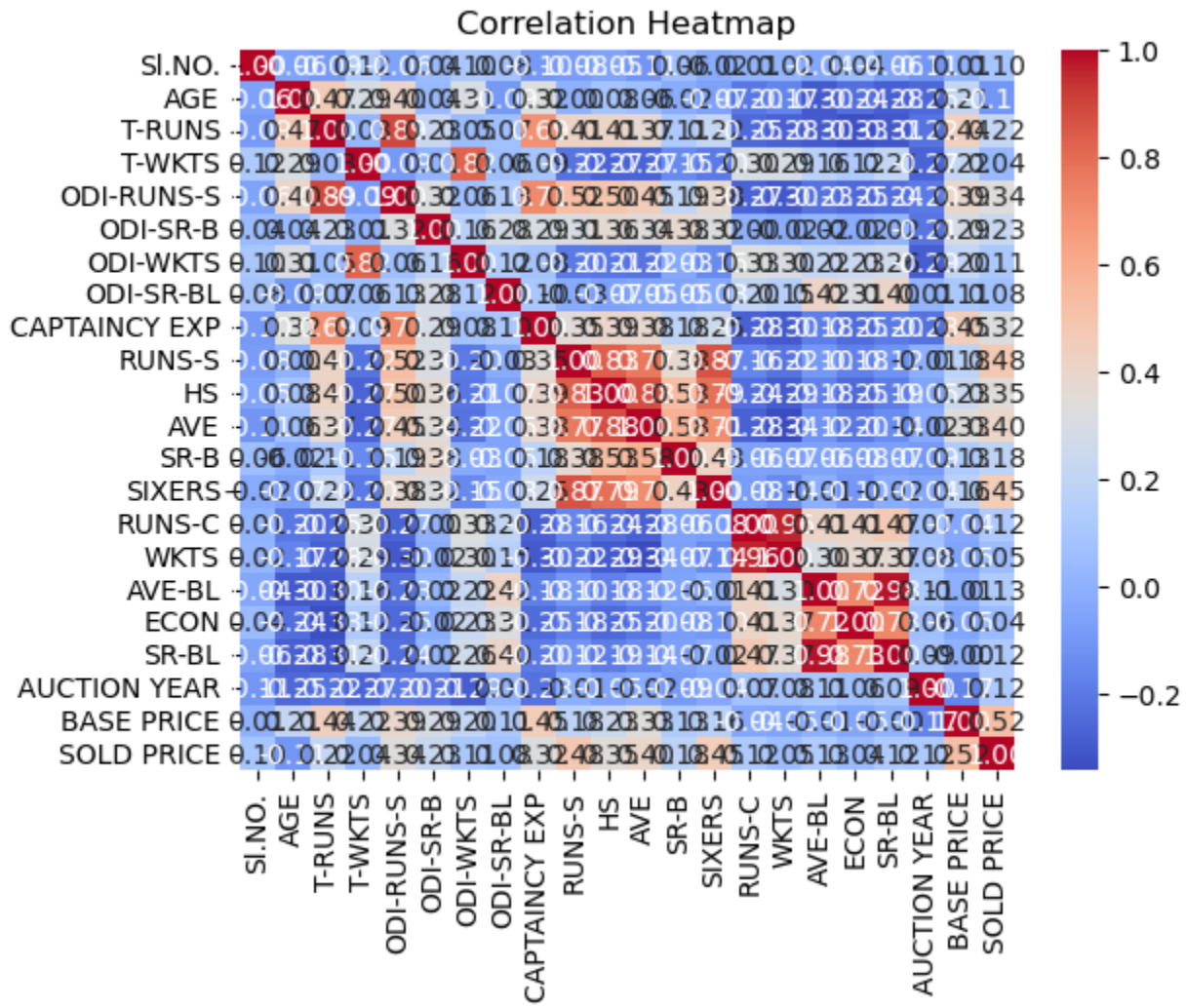
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Predict and calculate RMSE
train_pred = model.predict(X_train)
test_pred = model.predict(X_test)

print("Train RMSE:", np.sqrt(mean_squared_error(y_train, train_pred)))
print("Test RMSE:", np.sqrt(mean_squared_error(y_test, test_pred)))
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF




```
Train RMSE: 246632.92817418306
Test RMSE: 269568.3866969563
```

In [16]:

```
# ----------------------Task 9---------------------
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression

emails = [
    "Win a free iPhone now click this link",
    "Hi team, let's schedule the project meeting for tomorrow",
    "Claim your lottery prize of 100000 dollars",
    "Please find attached the quarterly financial report",
    "Limited time offer buy one get one free",
    "Can we catch up for coffee this evening?"
]

labels = [1, 0, 1, 0, 1, 0]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(emails)

model = LogisticRegression()
model.fit(X, labels)

new_email = ["Congratulations you won a cash prize"]
X_new = vectorizer.transform(new_email)
prediction = model.predict(X_new)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
print("Email:", new_email[0])
print("Folder to move:", "Spam Folder" if prediction[0] == 1 else "Inbox")
```

```
Email: Congratulations you won a cash prize
Folder to move: Spam Folder
```

In [1]:

```
# ------------------------------Task 10---------------------------------
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load dataset
df = pd.read_csv(r"C:\Users\hanis\OneDrive\Desktop\Hanish\DAA\glass.csv")

# Separate input and output
X = df.drop('Type', axis=1)
y = df['Type']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Decision Tree using Gini Index
model1 = DecisionTreeClassifier(criterion='gini', random_state=42)
model1.fit(X_train, y_train)
pred1 = model1.predict(X_test)

print("Gini Index Accuracy:",
      round(accuracy_score(y_test, pred1) * 100, 2), "%")

# Decision Tree using Entropy
model2 = DecisionTreeClassifier(criterion='entropy', random_state=42)
model2.fit(X_train, y_train)
pred2 = model2.predict(X_test)

print("Entropy Measure Accuracy:",
      round(accuracy_score(y_test, pred2) * 100, 2), "%")
```

```
Gini Index Accuracy: 72.09 %
Entropy Measure Accuracy: 76.74 %
```

In [3]:

```
#----------------------- task 11---------------------------------
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# 1. Load dataset
df = pd.read_csv(r"C:\Users\hanis\OneDrive\Desktop\Hanish\DAA\glass.csv")

# 2. Separate features and target
X = df.drop('Type', axis=1)
y = df['Type']
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```python
# 3. Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 4. Train Random Forest Classifier
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

# 5. Predict and calculate accuracy
y_pred = model.predict(X_test)
print("Accuracy:", round(accuracy_score(y_test, y_pred) * 100, 2), "%")

# 6. Display Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

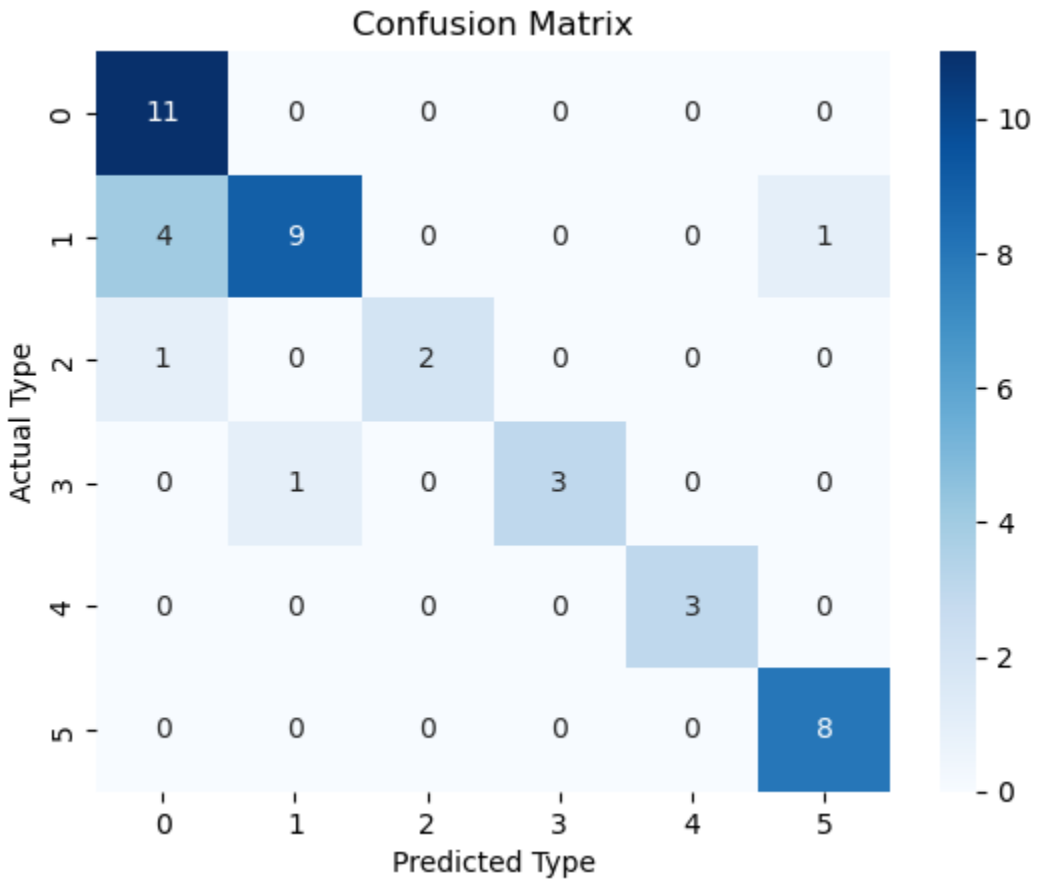
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix")
plt.xlabel("Predicted Type")
plt.ylabel("Actual Type")
plt.show()

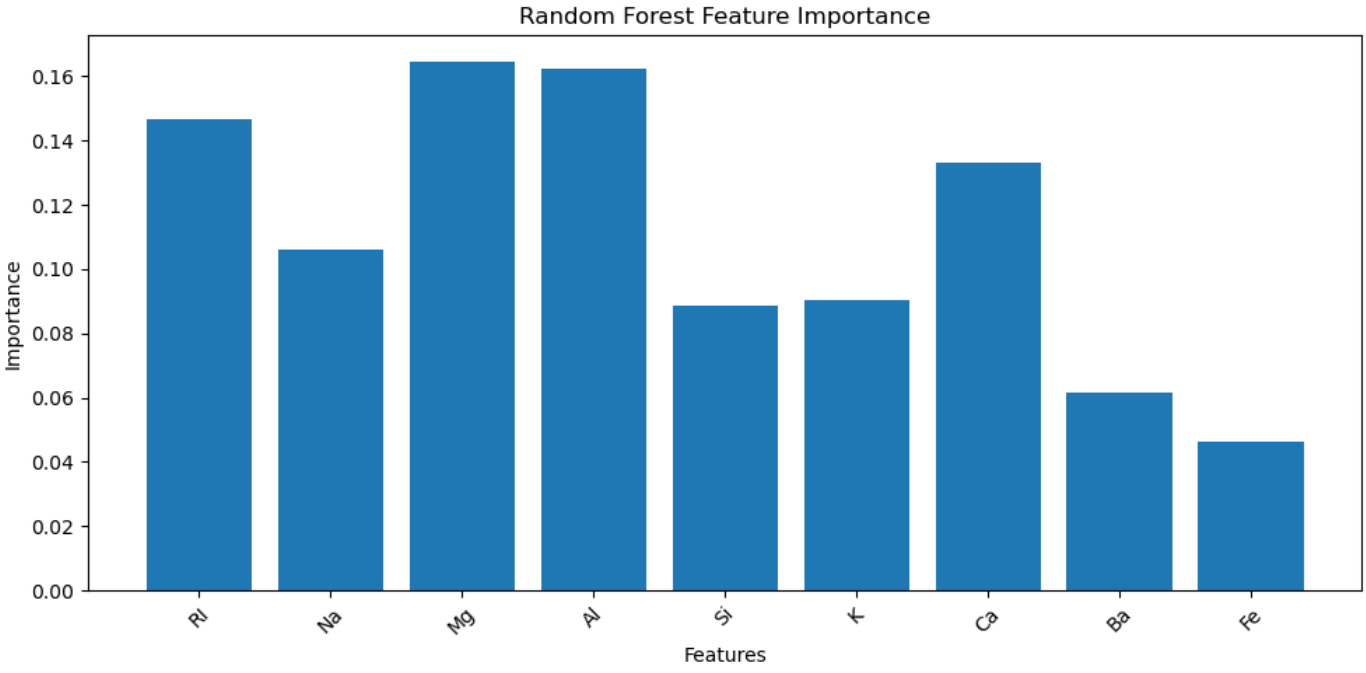
# 7. Plot Feature Importance
importance = model.feature_importances_

plt.figure(figsize=(10, 5))
plt.bar(X.columns, importance)
plt.title("Random Forest Feature Importance")
plt.xlabel("Features")
plt.ylabel("Importance")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
Accuracy: 83.72 %
Confusion Matrix:
 [[11  0  0  0  0  0]
 [ 4  9  0  0  0  1]
 [ 1  0  2  0  0  0]
 [ 0  1  0  3  0  0]
 [ 0  0  0  0  3  0]
 [ 0  0  0  0  0  8]]
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF





In [7]:

```python
# -------------------------Task 12------------------
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

# 1. Generate sample data points for clustering
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=42)

# 2. Implement K-Means Clustering (setting K = 4)[cite: 11]
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

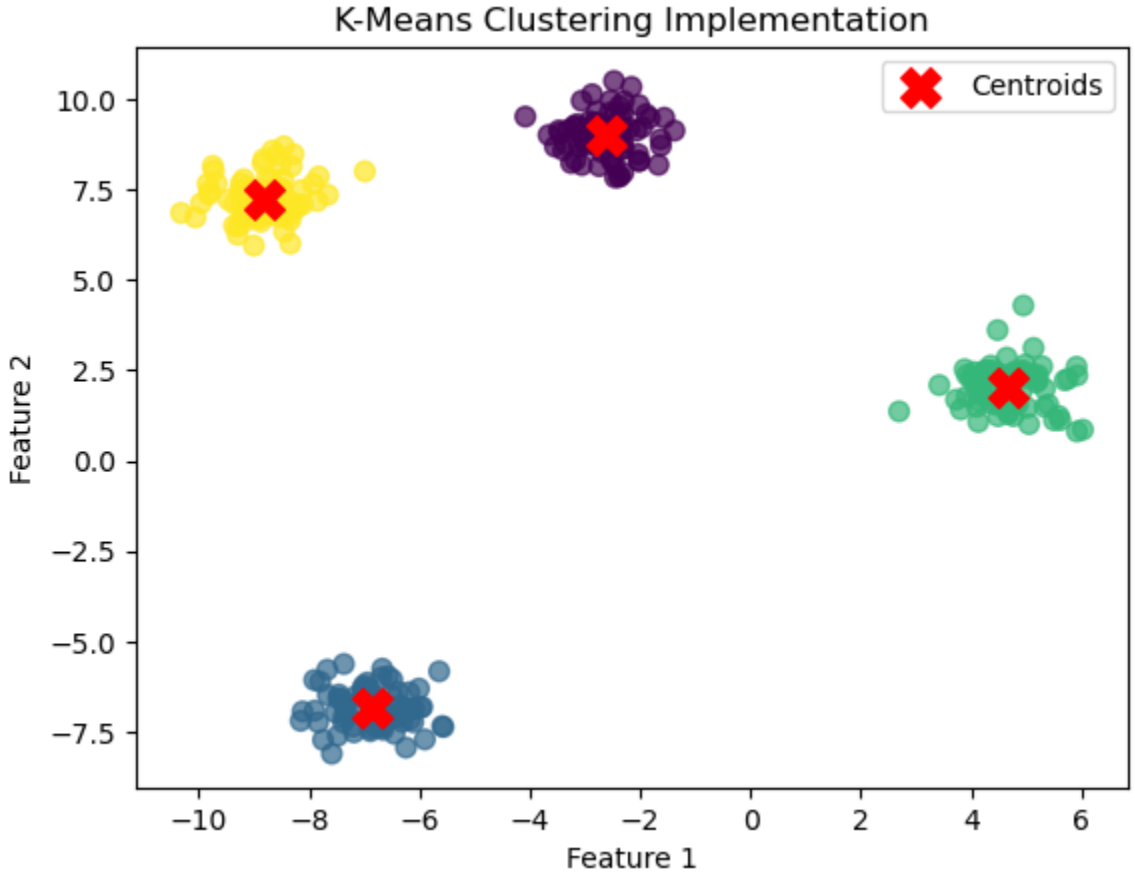
```
kmeans.fit(X)

# Get cluster centers and labels assigned to each point
centroids = kmeans.cluster_centers_
labels = kmeans.labels_

print("Cluster Centers:\n", centroids)

# 3. Visualize the clusters and centroids
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', s=50, alpha=0.7)
plt.scatter(centroids[:, 0], centroids[:, 1], c='red', marker='X', s=200, label='Centroi
plt.title("K-Means Clustering Implementation")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.show()
```

```
C:\Users\hanis\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks th
an available threads. You can avoid it by setting the environment variable OMP_NUM_THREA
DS=2.
  warnings.warn(
Cluster Centers:
 [[-2.60516878  8.99280115]
 [-6.85126211 -6.85031833]
 [ 4.68687447  2.01434593]
 [-8.83456141  7.24430734]]
```



In [ ]:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF